

Cryptography - 3_10

Proof

By definition:

$$SD(Y; U) = \frac{1}{2} \sum_{y \in \mathcal{Y}} |Pr[Y = y] - Pr[U = y]| = \frac{1}{2} \sum_{y \in \mathcal{Y}} |Pr[Y = y] - \frac{1}{|\mathcal{Y}|}|$$

it's uniform, U defined over $\mathcal{Y}$

Let $q_y = Pr[Y = y] - \frac{1}{|\mathcal{Y}|}$ and

$$s_y = \begin{cases} 1 & \text{if } q_y > 0 \\ -1 & \text{else} \end{cases}$$

Hence,

$$SD(Y; U) = \frac{1}{2} \sum_{y \in \mathcal{Y}} q_y \cdot s_y =$$

We then write them as vectors, with $\vec{q} = (q_y)_{y \in \mathcal{Y}}$ and $\vec{s} = (s_y)_{y \in \mathcal{Y}}$

$$= \frac{1}{2} \langle \vec{q}, \vec{s} \rangle \underset{*_1}{\leq} \frac{1}{2} \sqrt{\langle \vec{q}, \vec{q} \rangle \cdot \langle \vec{s}, \vec{s} \rangle} =$$

*₁ By Cauchy-Schwarz, famous inequality

$$= \frac{1}{2} \sqrt{\sum_{y \in \mathcal{Y}} q_y^2 \cdot |\mathcal{Y}|}$$

Now we analyze term $\sum_{y \in \mathcal{Y}} q_y^2$.

$$\begin{aligned} \sum_{y \in \mathcal{Y}} q_y^2 &= \sum_{y \in \mathcal{Y}} \left(Pr[Y = y] - \frac{1}{|\mathcal{Y}|} \right)^2 = \sum_{y \in \mathcal{Y}} \left(Pr[Y = y]^2 + \frac{1}{|\mathcal{Y}|^2} - 2 \frac{Pr[Y = y]}{|\mathcal{Y}|} \right) = \\ &= \underbrace{\sum_{y \in \mathcal{Y}} Pr[Y = y]}_{Col(Y)} + \frac{1}{|\mathcal{Y}|} - \frac{2}{|\mathcal{Y}|} = Col(Y) - \frac{2}{|\mathcal{Y}|} \underset{*_2}{=} \end{aligned}$$

*₂ we use the definition of $Col(Y)$, and also $\sum_{y \in \mathcal{Y}} Pr[Y = y] = 1$.

$$\leq \frac{1}{|\mathcal{Y}|} (1 + 4\epsilon^2) - \frac{1}{|\mathcal{Y}|} = \frac{4\epsilon^2}{|\mathcal{Y}|}$$

Then:

$$SD(Y; U) \leq \frac{1}{2} \sqrt{\frac{4\mathcal{E}^2}{|\mathcal{Y}|}} = \mathcal{E}$$

We've then proven the Lemma.

■

Next, we apply the lemma to prove the theorem.

Proof

Let $Y = (S, \text{Ext}(S, X)) = (S, h(S, X))$.

If we take Y be the tuple, then all we need to do to prove the leftover hash lemma is that the statistical distance is at most $\mathcal{E}$. The lemma tells us that, in order to be true, we need to bound the collision probability, then the statistical distance will be at most $\mathcal{E}$.

Then we compute $\text{Col}(Y)$.

$$\text{Col}(Y) = \sum_{y \in \mathcal{Y}} \Pr[Y = y]^2 \stackrel{*_1}{=} \Pr[Y = Y'] =$$

$*_1$ where Y and Y' are two independent copies of the same random variable.

$$= \Pr[S = S' \wedge h(S, X) = h(S', X')] \stackrel{*_2}{=} \Pr[S = S' \wedge h(S, X) = h(S, X')] =$$

$*_2$ because $S = S'$.

Because S it's uniform and independent, we have

$$= \Pr[S = S'] \cdot \Pr[h(S, X) = h(S, X')] =$$

What is $\Pr[S = S']$? We know that it's equal to $\text{Col}(S)$, but considering that it's uniform, what's its value? 2^{-d}

$$= 2^{-d} \cdot \Pr[h(S, X) = h(S, X')] =$$

we then divide the second in two cases

$$= 2^{-d} (\Pr[X = X'] + \Pr[h(S, X) = h(S, X') \wedge X \neq X']) \leq$$

One case where $X = X'$ and one when it's not.

X now has entropy k , so we can bound the value of prob. to 2^{-k} .

2^{-l} comes from pairwise independent.

$$\leq 2^{-d}(2^{-k} + 2^{-l}) = \frac{1}{2^{d+l}}(2^{l-k} + 1) \leq$$

we use the assumption of the leftover hash lemma

$$\leq \frac{1}{2^{d+l}}(2^{2-2\log(1/\epsilon)} + 1) = \frac{1}{|\mathcal{Y}|} \cdot (4\epsilon^2 + 1)$$

We've seen how $Col(Y)$ is equal to the end value, which considering the lemma is exactly what we wanted to prove.

■

Computational Security

Summary of what we've seen so far in Information Theory:

We know that without any assumptions we can do symmetric crypto and randomness extraction generation with some strong limitations.

- Privacy: $|msg| = |Key|$ and one-time only.
- Integrity: Same as above.
- Randomness: we can't extract more than k from min-entropy k .

Next, we want to overcome all this limitations. We'll do so at the price of assumptions.

1. Adversary is computationally bounded
2. There exists hard problems

We will make conditional statements:

✓ Theorem

If problem X is hard (against efficient solvers), then crypto-system Π is secure (against efficient adversaries).

Consequence: If Π not secure, $\exists$ efficient solver for X .

Depending on what X is, the above could be ground breaking.

≡ Examples

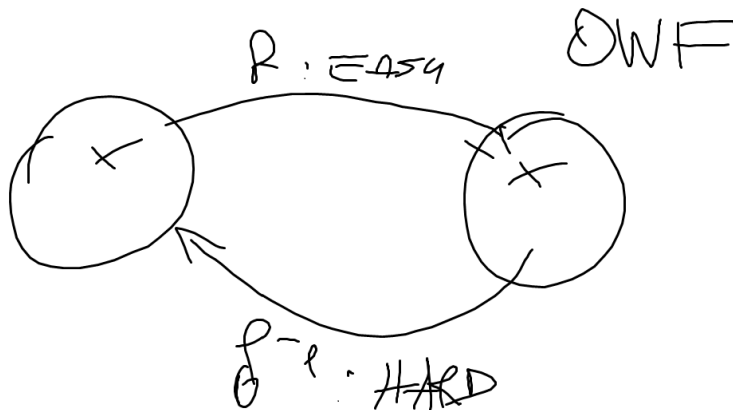
- $X = P \neq NP$.
- $X = \text{"Factoring is hard"}$

- $X = \text{"Discrete log is hard"}$
- ...

We are not able to just assume $P \neq NP$. (where P is the class of problems where there exist an efficient solver, while NP is the class where there exists an efficient verifier that takes a thing and check efficiently if it's a solution for the problem).

We will need "stronger" assumption:

One-way functions. These are functions f that are easy to compute but hard to invert:



Clearly, $OWF \implies P \neq NP$. Why? Because if $P = NP$, OWF do not exist as checking if $f(x) = y$ is efficient, and thus it's in $NP = P$.

We cannot exclude that $P \neq NP$, but still OWF do not exist.

To better understand this, we can refer to the following worlds (*yes, worlds where we could live, obviously we don't know in which we live*) considered by Russel Impagliazzo:

Impagliazzo:

- Algoritmica, $P = NP$
- Heuristica, $P \neq NP$ but no "average-hard puzzles", *we can think as OWF as a puzzle, we can generate a y that we cannot solve backwards. With an average hard puzzle we can generate an instance of a problem, without knowing the solution.*
- Pessiland, $P \neq NP$ but no OWF
- Minicrypt, $OWFs$ exist.
- Cryptomania, $OWFs$ + "public key crypto"

Great, we must start by fixing a modern computation: Turing machines. Efficient computation: Poly-time TMs.

Let's be generous: Adversaries can use any amount (polynomial) of randomness: PPT TMs (Probabilistic Polynomial Time Turing Machines).

In what comes next, one could follow two approaches:

1. Concrete security: Security holds with respect to t -time TMs except with probability $\leq \mathcal{E}$ for concrete parameters (like $t = 2^{20}$ steps, and $\mathcal{E} = 2^{-80}$). *Let's be super concrete, bringing concrete resources, with the adversary being a t -time Turing machine.*
2. Asymptotic Security. *We will actually follow this one.*

Let $\lambda \in \mathbb{N}$ be a security parameters. Adversaries now are $\text{poly}(\lambda)$ -time PPT TMs.

What about $\mathcal{E}$? We will call $\mathcal{E} = \text{negligible} = \text{negl}(\lambda)$.

✓ Definition (negligible)

$\mathcal{E} : \mathbb{N} \rightarrow \mathbb{R}$ is negligible if

$\forall p(\lambda) = \text{poly}(\lambda)$ (polynomial)

$\exists \lambda_0 \in \mathbb{N}$ such that $\forall \lambda > \lambda_0$ we have $\mathcal{E}(\lambda) < 1/p(\lambda)$.

In other words, $\mathcal{E}(\lambda) \leq O(\frac{1}{p(\lambda)}) \quad \forall p(\lambda) = \text{poly}(\lambda)$.